PHYB57 · CHAPTER 17 · 第 17 章

插值 · 全面复习讲义

Interpolation · Comprehensive Review Notes

中英双语 · KaTeX 公式 · 深色阅读版

Bilingual · KaTeX formulas · dark reading edition

KaTeX 公式 · KaTeX | 中英对照 · Bilingual | 每页考点速记 · Key points on every page | 按幻灯片顺序 · Slide order | 拓展 + 6 道练习 · Extensions + 6 problems

覆盖 C17 全部 10 页幻灯片：插值问题、线性插值、三次样条（条件与边界）、SciPy 实现、Lagrange 多项式插值、Newton 多项式插值与差商表、Python 实现。

> Covers all 10 slides of C17: the interpolation problem, linear interpolation, cubic splines (conditions and boundaries), the SciPy implementation, Lagrange polynomial interpolation, Newton polynomial interpolation with divided-difference tables, and the Python code.

另加幻灯片没有展开的内容（标 拓展 EXT）：线性插值误差界、方程计数、样条手算、插值多项式唯一性、Runge 现象与外推风险、SciPy 新版 API；最后是公式总表和 6 道完整解答的练习题。所有数值都用 Python 复算过，幻灯片中的两处错误已在原处标出。

> Added material the slides do not develop (tagged 拓展 EXT): the linear-interpolation error bound, equation counting, a spline solved by hand, uniqueness of the interpolating polynomial, Runge's phenomenon and extrapolation risk, and current SciPy APIs; then a formula sheet and 6 fully worked problems. Every number was recomputed in Python; two errors in the slides are flagged where they occur.

PHYB57 复习用 · 2026 年 11 月 5 日

> For PHYB57 review · 5 November 2026

考点速记 KEY POINTS

- 插值：$\hat{f}(x_i) = y_i$，曲线必须过每个数据点。
  Interpolation: $\hat{f}(x_i) = y_i$, through every point.
- Lagrange：$L(x) = \sum_i y_i P_i(x)$。
  Lagrange: $L(x) = \sum_i y_i P_i(x)$.
- 三次样条：$n-1$ 段，$4(n-1)$ 个未知数。
  Cubic spline: $n-1$ pieces, $4(n-1)$ unknowns.
- Newton：系数 = 差商表第一行。
  Newton: coefficients = top row of the divided-difference table.

本章地图与记号

Chapter map and notation

节 Sec.	内容 Content	幻灯片 Slides
1	插值问题 · The interpolation problem	1
2	线性插值 · Linear interpolation +误差界 error bound	2
3	三次样条：定义、条件、边界 · Cubic splines: definition, conditions, boundaries +计数 counting	3–4
4	SciPy 实现样条 · Splines in SciPy +bc_type	5
5	Lagrange 多项式插值 · Lagrange polynomial interpolation +唯一性 uniqueness	6–7
6	Newton 多项式插值与差商 · Newton interpolation and divided differences	8–10
7	方法比较、Runge 现象、外推 · Comparison, Runge's phenomenon, extrapolation 拓展 EXT	—
—	公式总表 · 练习 Q1–Q6 · 答案与来源 · Formula sheet · Problems · Key & sources	—

记号约定 / Notation

数据与插值函数 / data and interpolant

$$(x_i, y_i),\ i = 1, \dots, n, \qquad \hat{f}(x_i) = y_i$$

区间长度 / interval width

$$h_i = x_{i+1} - x_i, \qquad x_1 < x_2 < \cdots < x_n$$

- 幻灯片中样条和 Lagrange 的下标从 1 开始（$x_1, \dots, x_n$），Newton 的下标从 0 开始（$x_0, \dots, x_n$）。本讲义沿用各自的写法，阅读时注意点数：n 个点 ↔ 次数 $\le n-1$。
  > The slides index splines and Lagrange from 1 ($x_1, \dots, x_n$) and Newton from 0 ($x_0, \dots, x_n$). These notes keep each convention; watch the point count: n points ↔ degree $\le n-1$.
- 插值（interpolation）：在数据范围内估值；外推（extrapolation）：在范围外估值，风险大得多。
  > Interpolation: estimating inside the data range; extrapolation: estimating outside it, which is far riskier.
- 反复使用的例子：幻灯片数据 $x = [0, 1, 2]$，$y = [1, 3, 2]$。
  > Running example: the slides' data $x = [0, 1, 2]$, $y = [1, 3, 2]$.
- 标 拓展 EXT 的内容超出幻灯片。
  > Items tagged 拓展 EXT go beyond the slides.

考点速记 KEY POINTS

- n 个点唯一确定一个次数 $\le n-1$ 的多项式。
  n points fix a unique polynomial of degree $\le n-1$.
- 样条/Lagrange 下标从 1，Newton 从 0。
  Splines/Lagrange index from 1, Newton from 0.
- 插值 ≠ 回归：插值必须过每个点。
  Interpolation ≠ regression: it hits every point.
- 外推不可靠。
  Extrapolation is unreliable.

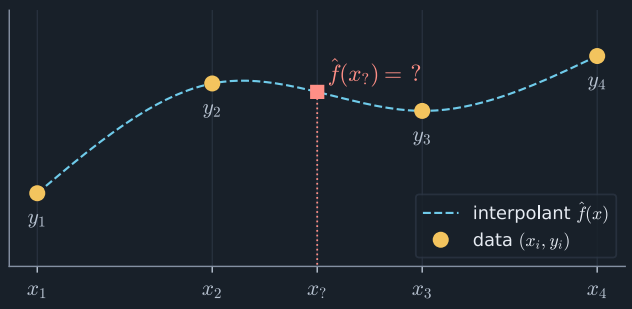
1 插值问题

1 Interpolation problem statement

概述 / Overview

插值的定义：估计落在已知数据点范围之内的未知值的过程。

> Interpolation definition: the process of estimating unknown values that fall within the range of known data points.



已知 (x_i, y_i)，求 $x_?$ 处的值 / given (x_i, y_i), find the value at $x_?$

问题陈述 / Problem statement

- 给定一组自变量数据 x_i 和因变量数据 y_i，目标是找到估计函数 $\hat{f}(x)$，使数据集中的每个点都满足 $\hat{f}(x_i) = y_i$。
  > Given independent data values x_i and dependent data values y_i, the goal is to find an estimation function $\hat{f}(x)$ such that $\hat{f}(x_i) = y_i$ for every point in the dataset.
- 这个函数应当穿过所有数据点，从而对数据集中没有的新 x 值给出准确估计。
  > This function should pass through all the data points, giving accurate estimates for new x values not explicitly in the dataset.

$$\text{find } \hat{f} \text{ such that } \hat{f}(x_i) = y_i \quad \text{for all } i = 1, \ldots, n$$

主要特点 / Key characteristics

- 与回归不同，插值不对数据假设模型。
  > Unlike regression, interpolation does not assume a model for the data.
- 适用于数据点可靠、能准确反映趋势的情况。
  > Suitable when data points are reliable and accurately represent the trend.
- 例：估计汽车随时间的运动。数据点是光滑的，反映了运动的连续性。
  > Example: estimating the motion of a car over time, where the data points are smooth and reflect the continuous nature of the movement.

插值 vs 回归（C16） / Interpolation vs regression (C16)

回归（最小二乘）选定模型、允许曲线不过数据点，适合**有噪声**的数据；插值强制过每个点，数据有噪声时会把噪声也"插"进去。判断题常考：数据有测量误差 → 选回归。

> Regression (least squares) chooses a model and lets the curve miss the points, which suits **noisy** data; interpolation forces the curve through every point and so reproduces the noise too. A common exam question: data with measurement error → use regression.

考点速记 KEY POINTS

- 插值条件 $\hat{f}(x_i) = y_i$。
  Interpolation condition $\hat{f}(x_i) = y_i$.
- 只在数据范围内估值。
  Estimate only inside the data range.
- 不假设模型；回归才假设模型。
  No assumed model; regression assumes one.
- 噪声数据用回归，不用插值。
  Noisy data: regression, not interpolation.

2 线性插值

2 Linear interpolation

定义：假设待估点位于连接其两侧最近数据点的直线上。

> Definition: assumes the estimated point lies on the straight line joining the nearest data points on either side.

对 $x_i < x < x_{i+1}$，线性插值公式为：

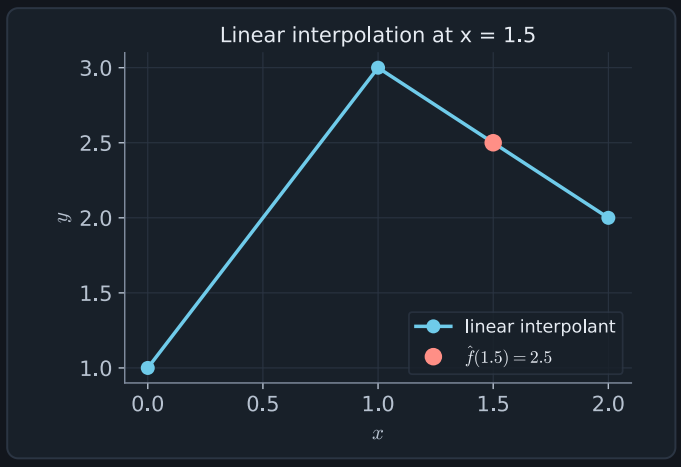
> For data points $x_i < x < x_{i+1}$, the linear interpolation formula is:

$$\hat{f}(x) = y_i + \frac{(y_{i+1} - y_i)(x - x_i)}{x_{i+1} - x_i}$$

例：$x = [0, 1, 2]$，$y = [1, 3, 2]$，求 $x = 1.5$ 处的插值。1.5 落在 $[1, 2]$，用 $(1, 3)$ 与 $(2, 2)$：

> Example: given $x = [0, 1, 2]$, $y = [1, 3, 2]$, find the interpolation at $x = 1.5$. Since $1.5 \in [1, 2]$, use $(1, 3)$ and $(2, 2)$:

$$\hat{f}(1.5) = 3 + \frac{(2 - 3)(1.5 - 1)}{2 - 1} = 2.5$$



Python 实现 / Practical implementation

```
from scipy.interpolate import interp1d
import matplotlib.pyplot as plt

plt.style.use('seaborn-poster')
```

```
x = [0, 1, 2]
y = [1, 3, 2]

f = interp1d(x, y)
y_hat = f(1.5)
print(y_hat)
```

输出 2.5。注意：interp1d 默认不允许外推，f(3) 会报 ValueError；np.interp(3, x, y) 则返回端点值 2.0（把范围外"夹"在端点）。SciPy 文档已把 interp1d 标为 legacy（不再更新）。'seaborn-poster' 在新版 Matplotlib 中需写 'seaborn-v0_8-poster'。

> It prints 2.5. Note: interp1d refuses to extrapolate by default, so f(3) raises ValueError; np.interp(3, x, y) instead returns the endpoint value 2.0 (it clamps outside the range). The SciPy docs mark interp1d as legacy (no further updates). In current Matplotlib, 'seaborn-poster' must be written 'seaborn-v0_8-poster'.

误差界 拓展 EXT / **Error bound**

在长为 h 的区间上 $|f(x) - \hat{f}(x)| \le \frac{h^2}{8} \max |f''|$：误差 $O(h^2)$，点距减半误差约变 1/4。例：$\sin x$、$h = 0.5$，实测最大误差 $0.0306 \le 0.03125$。

> On an interval of width h, $|f(x) - \hat{f}(x)| \le \frac{h^2}{8} \max |f''|$: error $O(h^2)$, so halving the spacing roughly quarters it. Example: $\sin x$ with $h = 0.5$ gives a measured maximum error $0.0306 \le 0.03125$.

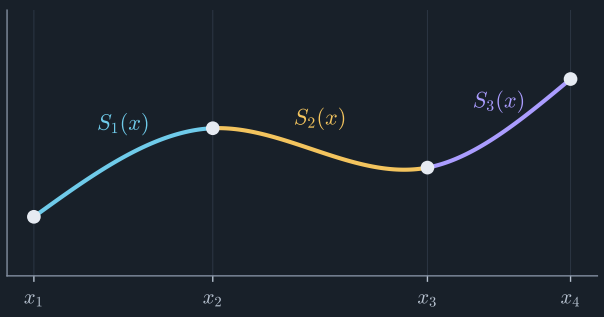
考点速记 KEY POINTS

- 先找区间 $x_i < x < x_{i+1}$，再代公式。
  Find the interval $x_i < x < x_{i+1}$, then apply the formula.
- 误差 $\le \frac{h^2}{8} \max |f''|$。
  Error $\le \frac{h^2}{8} \max |f''|$.
- $\hat{f}(1.5) = 2.5$（幻灯片例）。
  $\hat{f}(1.5) = 2.5$ (slide example).
- interp1d 越界报错，np.interp 取端点值。
  interp1d errors out of range; np.interp clamps.

3 三次样条插值

3 Cubic spline interpolation

- 定义：用分段三次多项式构造一条穿过一组点的光滑曲线。
  > Definition: a method of constructing a smooth curve through a set of points using piecewise cubic polynomials.
- 目标：曲线不仅要穿过给定数据点，还要通过匹配一阶和二阶导数，使各点之间平滑过渡。
  > Objective: the curve must not only pass through the given data points but also transition smoothly between them by matching first and second derivatives.



4 个点 → 3 段三次多项式 S_1, S_2, S_3，在内点处光滑拼接 / 4 points → 3 cubic pieces S_1, S_2, S_3, joined smoothly at the interior points

数学表述 / Mathematical formulation

在每个区间 $[x_i, x_{i+1}]$ 上使用如下形式的三次多项式：

> For each interval between x_i and x_{i+1}, a cubic polynomial of the form below is used:

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i, \qquad x \in [x_i, x_{i+1}], \quad i = 1, \ldots, n-1$$

共有 $n-1$ 个三次函数，每个需要 4 个系数，所以总共有 $4(n-1)$ 个未知数。

> There are $n-1$ cubic functions to find, each needing four coefficients, giving a total of $4(n-1)$ unknowns.

手算技巧 拓展 EXT / **Hand-calculation tip**

手算时常把每段写成局部变量 $t = x - x_i$ 的形式：$S_i = a_i t^3 + b_i t^2 + c_i t + d_i$。这样 $S_i(x_i) = d_i$，$S_i'(x_i) = c_i$，$S_i''(x_i) = 2b_i$，方程会简单得多。两种写法描述同一个样条，只是系数不同；SciPy 的 CubicSpline.c 存的就是局部形式的系数（见 Q5）。

> By hand, write each piece in the local variable $t = x - x_i$: $S_i = a_i t^3 + b_i t^2 + c_i t + d_i$. Then $S_i(x_i) = d_i$, $S_i'(x_i) = c_i$, $S_i''(x_i) = 2b_i$, and the equations become much simpler. Both forms describe the same spline with different coefficients; SciPy's CubicSpline.c stores the local-form coefficients (see Q5).

为什么用三次 / Why cubic

三次是能让 S、S'、S'' 在接点都连续的最低次数。每段次数低，所以不会像高次多项式那样剧烈振荡（见第 7 节 Runge 现象）。

> Cubic is the lowest degree that keeps S, S' and S'' continuous at the joins. Each piece has low degree, so it does not oscillate wildly as a high-degree polynomial can (see Runge's phenomenon, Section 7).

考点速记 KEY POINTS

- n 个点 → $n-1$ 段三次多项式。
  n points → $n-1$ cubic pieces.
- 接点处 S, S', S'' 连续。
  S, S', S'' continuous at the joins.
- 未知数 $4(n-1)$。
  Unknowns: $4(n-1)$.
- 手算用局部变量 $t = x - x_i$。
  By hand, use the local variable $t = x - x_i$.

3.2 三次样条的条件与约束

3.2 Conditions and constraints for cubic spline interpolation

建立方程 / Equation setup

- 插值条件：各段三次多项式必须经过给定数据点（每段两端各一个）。
  > Interpolation condition: the cubic polynomials must pass through the given data points (one at each end of every piece).
- 连续性：每段样条的一阶和二阶导数在数据点处必须连续，以保证平滑过渡。
  > Continuity: the first and second derivatives of each spline must be continuous at the data points to ensure smooth transitions.

插值 · $2(n-1)$ 个 / interpolation

$$S_i(x_i) = y_i, \quad S_i(x_{i+1}) = y_{i+1}, \quad i = 1, \ldots, n-1$$

连续性 · $2(n-2)$ 个 / continuity

$$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1})$$
$$S_i''(x_{i+1}) = S_{i+1}''(x_{i+1}) \quad i = 1, \ldots, n-2$$

边界条件 / Boundary conditions

- 常用约束是令两端点的二阶导数为零，称为自然样条（natural spline）条件。
  > A common constraint sets the second derivatives at the endpoints to zero, known as the natural spline condition.
- 其他边界条件可通过指定类型实现（SciPy CubicSpline 的 bc_type 参数）。
  > Alternative boundary conditions are set by specifying the type (the bc_type parameter of SciPy's CubicSpline).

$$S_1''(x_1) = 0, \qquad S_{n-1}''(x_n) = 0 \qquad \text{(natural spline, 2 equations)}$$

方程计数 拓展 EXT / Counting equations

$$\underbrace{2(n-1)}_{\text{interpolation}} + \underbrace{2(n-2)}_{\text{continuity}} + \underbrace{2}_{\text{boundary}} = 4n-4 = 4(n-1) \text{ unknowns } \checkmark$$

bc_type	两端条件 End conditions	说明 Note
'not-a-knot'	S''' 在 x_2、x_{n-1} 连续 · continuous S''' at x_2, x_{n-1}	**CubicSpline 的默认值 · CubicSpline's default**
'natural'	$S'' = 0$	幻灯片所用 · used on the slides
'clamped'	$S' = 0$	端点斜率为零 · zero end slopes
'periodic'	S', S'' 首尾相等 · S', S'' match at both ends	要求 $y_1 = y_n$ · needs $y_1 = y_n$

注意 / Caution

不写 bc_type 时 CubicSpline 用的是 'not-a-knot'，不是自然样条。同一组数据结果不同：$x = [0, 1, 2]$ 时 $S(1.5)$ 分别为 natural 2.781、not-a-knot 2.875、clamped 2.594。

> Without bc_type, CubicSpline uses 'not-a-knot', not the natural spline. The results differ for the same data: with $x = [0, 1, 2]$, $S(1.5)$ is 2.781 (natural), 2.875 (not-a-knot) and 2.594 (clamped).

考点速记 KEY POINTS

- 插值 $2(n-1)$ + 连续 $2(n-2)$ + 边界 2。
  Interpolation $2(n-1)$ + continuity $2(n-2)$ + boundary 2.
- 连续性只在内点 $x_2, \ldots, x_{n-1}$。
  Continuity only at interior points $x_2, \ldots, x_{n-1}$.
- 自然样条：两端 $S'' = 0$。
  Natural spline: $S'' = 0$ at both ends.
- CubicSpline 默认 'not-a-knot'。
  CubicSpline defaults to 'not-a-knot'.

4 用 Python 的 SciPy 库实现

4 Implementation using Python's SciPy library

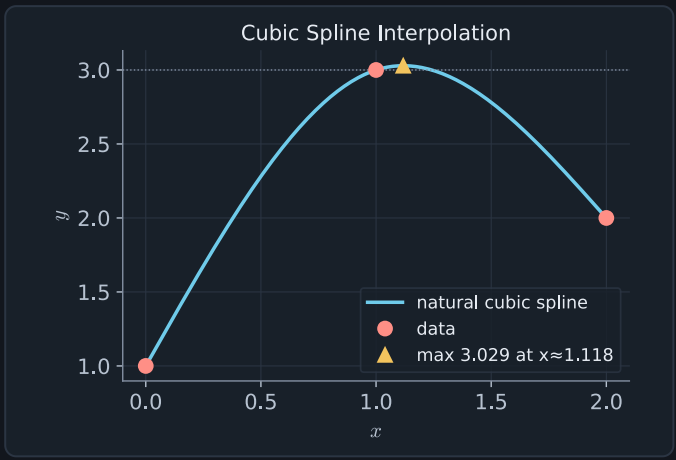
示例数据：$x = [0, 1, 2]$，$y = [1, 3, 2]$。

> Example data: $x = [0, 1, 2]$, $y = [1, 3, 2]$.

```
from scipy.interpolate import CubicSpline
import numpy as np
import matplotlib.pyplot as plt

x = [0, 1, 2]
y = [1, 3, 2]
f = CubicSpline(x, y, bc_type='natural')
x_new = np.linspace(0, 2, 100)
y_new = f(x_new)

plt.figure(figsize=(10,8))
plt.plot(x_new, y_new, 'b')
plt.plot(x, y, 'ro')
plt.title('Cubic Spline Interpolation')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```



说明：这段代码生成一条穿过指定点的光滑曲线，演示自然三次样条插值。linspace(0, 2, 100) 在 $[0, 2]$ 上取 100 个点（含两端）用于画图。

> Explanation: this code produces a smooth curve through the given points, demonstrating natural cubic spline interpolation. linspace(0, 2, 100) takes 100 points on $[0, 2]$ (both ends included) for plotting.

样条的系数 拓展 EXT / The spline's coefficients

$[0, 1]$ 段 / piece on $[0, 1]$

$$S_1(x) = -0.75x^3 + 2.75x + 1$$

$[1, 2]$ 段，$t = x - 1$ / piece on $[1, 2]$, $t = x - 1$

$$S_2 = 0.75t^3 - 2.25t^2 + 0.5t + 3$$

f.c 按列存这两组系数（最高次在上）。接点处 $S'(1) = 0.5$、$S''(1) = -4.5$ 两侧一致；两端 $S'' = 0$。曲线最大值 3.029（$x \approx 1.118$）略高于最大数据值 3：样条在数据点之间可能"冲过头"（overshoot）。

> f.c stores these coefficients column by column (highest power first). At the join, $S'(1) = 0.5$ and $S''(1) = -4.5$ agree on both sides; $S'' = 0$ at both ends. The curve peaks at 3.029 ($x \approx 1.118$), slightly above the largest data value 3: a spline can overshoot between data points.

考点速记 KEY POINTS

- CubicSpline(x, y, bc_type='natural')。
  CubicSpline(x, y, bc_type='natural').
- 本例自然样条 $S(1.5) = 2.78125$。
  Here the natural spline gives $S(1.5) = 2.78125$.
- 返回可调用对象：f(x_new)。
  Returns a callable: f(x_new).
- 样条可超过数据的最大值。
  A spline can exceed the data's maximum.

5 Lagrange 多项式插值

5 Lagrange polynomial interpolation

- Lagrange 多项式插值是找**一个**穿过所有给定数据点的多项式（不再分段）。
  > Lagrange polynomial interpolation finds a **single** polynomial that passes through all the given data points (no pieces).
- 记作 $L(x)$，构造时使数据集中每个点都满足 $L(x_i) = y_i$。
  > It is written $L(x)$ and is constructed so that $L(x_i) = y_i$ for every point in the dataset.

Lagrange 基多项式 / Lagrange basis polynomials

插值多项式可写成 Lagrange 基多项式 $P_i(x)$ 的线性组合：

> The interpolating polynomial can be written as a linear combination of Lagrange basis polynomials $P_i(x)$:

基多项式 / basis polynomial

$$P_i(x) = \prod_{j=1,\, j\neq i}^{n} \frac{x - x_j}{x_i - x_j}$$

Lagrange 多项式 / Lagrange polynomial

$$L(x) = \sum_{i=1}^{n} y_i \, P_i(x)$$

每个基多项式都有如下性质（称为"克罗内克 δ"性质）：

> Each basis polynomial has the property below (the "Kronecker delta" property):

$$P_i(x_j) = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases} \qquad \Rightarrow \qquad L(x_j) = \sum_i y_i P_i(x_j) = y_j$$

分子在 x_j（$j \neq i$）处为零；在 x_i 处分子等于分母 / the numerator vanishes at x_j ($j \neq i$); at x_i it equals the denominator

主要性质 / Key properties

- Lagrange 多项式准确地穿过每个指定的数据点。
  > The Lagrange polynomial passes exactly through each of the specified data points.
- 只应在给定数据点的范围内使用，以免在区间外出现不切实际的增长。
  > It should only be used for interpolation within the range of the data points, to avoid unrealistic growth outside this interval.

唯一性 拓展 EXT / **Uniqueness**

n 个不同节点确定唯一一个次数 $\leq n-1$ 的插值多项式：若两个这样的多项式都过这些点，它们的差有 n 个根但次数 $\leq n-1$，只能恒为零。所以 Lagrange、Newton 和 np.polyfit(x, y, n–1) 得到的是**同一个多项式**，只是写法不同。另外 $\sum_i P_i(x) \equiv 1$（取所有 $y_i = 1$ 即得）。

> n distinct nodes determine a unique interpolating polynomial of degree $\leq n-1$: if two such polynomials pass through the points, their difference has n roots but degree $\leq n-1$, so it is identically zero. Lagrange, Newton and np.polyfit(x, y, n–1) therefore give **the same polynomial**, written differently. Also $\sum_i P_i(x) \equiv 1$ (take every $y_i = 1$).

考点速记 KEY POINTS

- $P_i = \prod_{j\neq i} \frac{x-x_i}{x_i-x_j}$。
  $P_i = \prod_{j\neq i} \frac{x-x_j}{x_i-x_j}$.
- $P_i(x_j) = \delta_{ij}$：自己的点为 1，别的点为 0。
  $P_i(x_j) = \delta_{ij}$: 1 at its own node, 0 at the others.
- n 点 → 次数 $\leq n-1$，且唯一。
  n points → degree $\leq n-1$, and unique.
- $\sum_i P_i(x) = 1$。
  $\sum_i P_i(x) = 1$.

5.2 Lagrange 插值的示例与图像

5.2 Example and visualisation of Lagrange interpolation

数据点 $x=[0,1,2]$，$y=[1,3,2]$。由此得到的 Lagrange 基多项式：

> Consider the data points $x=[0,1,2]$, $y=[1,3,2]$. The Lagrange basis polynomials for this dataset are:

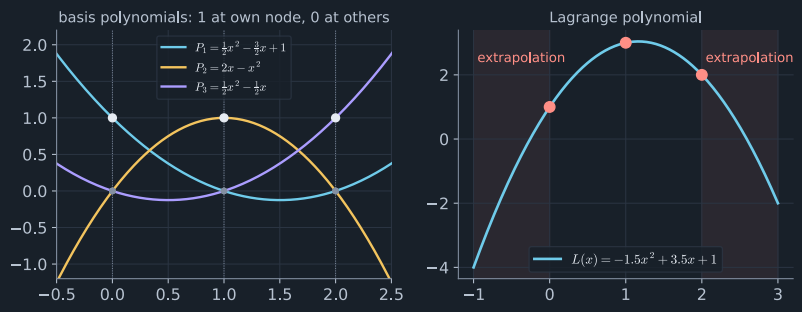
$$P_1(x)=\frac{(x-1)(x-2)}{(0-1)(0-2)}=\tfrac{1}{2}x^2-1.5x+1$$
$$P_2(x)=\frac{(x-0)(x-2)}{(1-0)(1-2)}=2x-x^2$$
$$P_3(x)=\frac{(x-0)(x-1)}{(2-0)(2-1)}=\tfrac{1}{2}x^2-0.5x$$

编者注：幻灯片有两处错误 / Editorial note: two errors on the slide

① 幻灯片写 $P_1(x)=-1.5x+0.5x^2$，漏了常数项 $+1$：这样 $P_1(0)=0$，违背 $P_1(x_1)=1$。教材 §17.4 写的是 $\frac{1}{2}(x^2-3x+2)$，正确。② 幻灯片说"这些多项式之和"构成 $L(x)$，应为**加权和** $L=\sum y_iP_i=1\cdot P_1+3P_2+2P_3$；普通的和 $\sum P_i\equiv 1$。

> ① The slide writes $P_1(x)=-1.5x+0.5x^2$, dropping the constant $+1$: then $P_1(0)=0$, violating $P_1(x_1)=1$. Textbook §17.4 has the correct $\frac{1}{2}(x^2-3x+2)$. ② The slide says "the sum of these polynomials" forms $L(x)$; it is the **weighted sum** $L=\sum y_iP_i=1\cdot P_1+3P_2+2P_3$; the plain sum is $\sum P_i\equiv 1$.

$$L(x)=1\cdot P_1+3P_2+2P_3=-1.5x^2+3.5x+1,\qquad L(0)=1,\ L(1)=3,\ L(2)=2\ \checkmark$$



图像：每个基多项式 $P_i(x)$ 在自己对应的数据点取 1，在其余点取 0；它们的加权和就是过 $(0,1)$、$(1,3)$、$(2,2)$ 的完整 Lagrange 多项式。区间外抛物线迅速下落：$L(-1)=-4$，$L(3)=-2$。

> Visualisation: each basis polynomial $P_i(x)$ equals 1 at its own data point and 0 at the others; their weighted sum is the full Lagrange polynomial through $(0,1)$, $(1,3)$ and $(2,2)$. Outside the interval the parabola drops fast: $L(-1)=-4$, $L(3)=-2$.

```
from scipy.interpolate import lagrange

f = lagrange(x, y)
```

SciPy 的 lagrange 直接算出 Lagrange 多项式（poly1d 对象，f.coef 为 $[-1.5,\ 3.5,\ 1]$），不必手算。**SciPy 1.18 起它已弃用，将在 1.20 删除**，改用 BarycentricInterpolator。

> SciPy's lagrange computes the Lagrange polynomial directly (a poly1d; f.coef is $[-1.5,\ 3.5,\ 1]$), no manual work needed. **It is deprecated from SciPy 1.18 and will be removed in 1.20**; use BarycentricInterpolator.

考点速记 KEY POINTS

- $P_1=\frac{1}{2}x^2-1.5x+1$（幻灯片漏了 $+1$）。
  $P_1=\frac{1}{2}x^2-1.5x+1$ (the slide drops the $+1$).
- $L=\sum y_iP_i$ 是加权和。
  $L=\sum y_iP_i$ is a weighted sum.
- $L(x)=-1.5x^2+3.5x+1$。
  $L(x)=-1.5x^2+3.5x+1$.
- 检验：P_i 在自己的点为 1。
  Check: each P_i is 1 at its own node.

6 Newton 多项式插值

6 Newton's polynomial interpolation

Newton 多项式插值是让多项式穿过一组数据点的方法，在数值分析中广泛使用。

> Newton's polynomial interpolation fits a polynomial through a set of data points and is widely used in numerical analysis.

公式 / Formula

n 次 Newton 多项式（用 $n+1$ 个点 $x_0, \ldots, x_n$）可写成：

> The Newton polynomial of order n (using $n+1$ points $x_0, \ldots, x_n$) can be written as:

$$f(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1)\cdots(x - x_{n-1})$$

关键特点：系数 a_i 用差商求得；加入新数据点时只需在末尾**增加一项**，前面的系数不变，更新很方便。

> Key feature: the coefficients a_i come from divided differences, so adding a data point only **appends one term** and leaves the earlier coefficients unchanged: easy updates.

逐点代入求系数 拓展 EXT / Coefficients by substitution

在 $x = x_0$ 处除第一项外都为零，依次代入可解出系数（下三角方程组）：

> At $x = x_0$ every term but the first vanishes; substituting point by point solves for the coefficients (a lower-triangular system):

$$a_0 = y_0, \qquad a_1 = \frac{y_1 - y_0}{x_1 - x_0} = f[x_0, x_1], \qquad a_2 = f[x_0, x_1, x_2],\ \ldots,\ a_k = f[x_0, \ldots, x_k]$$

幻灯片数据 $x = [0, 1, 2]$，$y = [1, 3, 2]$：$a = [1,\ 2,\ -1.5]$，$f(x) = 1 + 2x - 1.5x(x-1) = -1.5x^2 + 3.5x + 1$，与 Lagrange 结果相同（唯一性）。

> With the slide data $x = [0, 1, 2]$, $y = [1, 3, 2]$: $a = [1,\ 2,\ -1.5]$ and $f(x) = 1 + 2x - 1.5x(x-1) = -1.5x^2 + 3.5x + 1$, the same as Lagrange (uniqueness).

嵌套（Horner）求值 / Nested (Horner) evaluation

$$f(x) = a_0 + (x - x_0)\Big(a_1 + (x - x_1)\big(a_2 + \cdots + (x - x_{n-1})\,a_n\big)\Big)$$

只需 n 次乘法；第 6.3 节的 newton_poly 就是这样从内往外算

/ only n multiplications; newton_poly in Section 6.3 works from the inside out

考点速记 KEY POINTS

- $n+1$ 个点 → n 次 Newton 多项式。
  $n+1$ points → Newton polynomial of order n.
- 加一个点只加一项。
  One new point adds one term.
- $a_k = f[x_0, \ldots, x_k]$。
  $a_k = f[x_0, \ldots, x_k]$.
- 与 Lagrange 是同一个多项式。
  Same polynomial as Lagrange.

6.2 差商与系数计算

6.2 Divided differences and coefficient calculation

- 差商（divided differences）：计算多项式系数的方法。每个系数都由前一步的结果算出，所以效率高。

  > Divided differences: a method to calculate the polynomial's coefficients. Each coefficient is built from previous results, which makes it efficient.

- 迭代过程：差商可以写成：

  > Iterative process: the divided differences can be written as:

一阶 / first

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

二阶 / second

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$$

$$f[x_i, \ldots, x_{i+j}] = \frac{f[x_{i+1}, \ldots, x_{i+j}] - f[x_i, \ldots, x_{i+j-1}]}{x_{i+j} - x_i} \qquad \text{(general rule)}$$

分母是这一组**两端**的 x 之差，不是相邻点之差

/ the denominator is the difference of the **outermost** x values, not of neighbours

差商表（幻灯片 10 的数据） / Divided-difference table (slide 10 data)

差商表：系统计算多项式所需系数的表格。数据 $x = [-5, -1, 0, 2]$，$y = [-2, 6, 1, 3]$：

> A divided-difference table organises the calculation of the coefficients. Data $x = [-5, -1, 0, 2]$, $y = [-2, 6, 1, 3]$:

x_i	$f[x_i]$	一阶 first	二阶 second	三阶 third
-5	-2	$\frac{6-(-2)}{-1-(-5)} = 2$	$\frac{-5-2}{0-(-5)} = -1.4$	$\frac{2-(-1.4)}{2-(-5)} = \frac{17}{35} \approx 0.4857$
-1	6	$\frac{1-6}{0-(-1)} = -5$	$\frac{1-(-5)}{2-(-1)} = 2$	
0	1	$\frac{3-1}{2-0} = 1$		
2	3			

$$p(x) = -2 + 2(x+5) - 1.4(x+5)(x+1) + \tfrac{17}{35}(x+5)(x+1)x$$

系数 = 表的第一行（绿色） / coefficients = the table's top row (green)

考点速记 KEY POINTS

- 第一行就是 $a_0, a_1, \ldots, a_n$。
  The top row is $a_0, a_1, \ldots, a_n$.
- n 个点：表有 n 列，第 j 列 $n - j$ 项。
  n points: n columns, column j has $n - j$ entries.
- 分母用两端 x：$x_{i+j} - x_i$。
  Denominator uses the outer x: $x_{i+j} - x_i$.
- 本例 $a = [-2, 2, -1.4, 17/35]$。
  Here $a = [-2, 2, -1.4, 17/35]$.

6.3 Newton 插值的 Python 实现

6.3 Implementation example in Python

用差商实现 Newton 插值的 Python 代码：

> Python implementation of Newton's interpolation using divided differences:

```python
def divided_diff(x, y):
    '''
    function to calculate the divided
    differences table
    '''
    n = len(y)
    coef = np.zeros([n, n])
    # the first column is y
    coef[:,0] = y

    for j in range(1,n):
        for i in range(n-j):
            coef[i][j] = \
              (coef[i+1][j-1] - coef[i][j-1]) \
              / (x[i+j]-x[i])

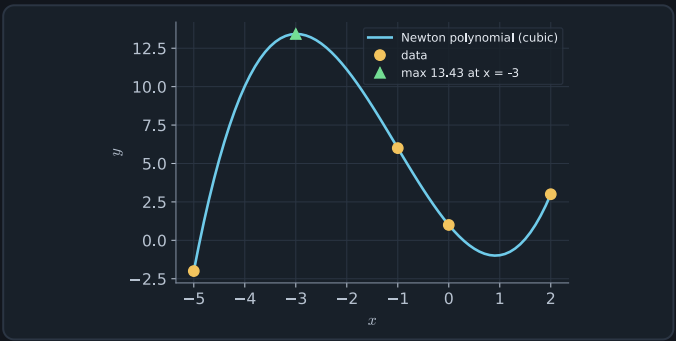
    return coef

def newton_poly(coef, x_data, x):
    '''
    evaluate the newton polynomial
    at x
    '''
    n = len(x_data) - 1
    p = coef[n]
    for k in range(1,n+1):
        p = coef[n-k] + (x -x_data[n-k])*p
    return p
```

```python
x = np.array([-5, -1, 0, 2])
y = np.array([-2, 6, 1, 3])
# get the divided difference coef
a_s = divided_diff(x, y)[0, :]

# evaluate on new data points
x_new = np.arange(-5, 2.1, .1)
y_new = newton_poly(a_s, x, x_new)

plt.figure(figsize = (12, 8))
plt.plot(x, y, 'bo')
plt.plot(x_new, y_new)
```



逐行理解 / Line by line

- coef[i][j] 存 $f[x_i, \ldots, x_{i+j}]$：第 0 列是 y，第 j 列用第 $j-1$ 列相邻两项相减，再除以 x[i+j] – x[i]。
  > coef[i][j] holds $f[x_i, \ldots, x_{i+j}]$: column 0 is y; column j subtracts neighbouring entries of column $j-1$ and divides by x[i+j] – x[i].
- divided_diff(x, y)[0, :] 取第一行，即系数 $a_0, \ldots, a_3 = [-2,\ 2,\ -1.4,\ 0.4857]$。
  > divided_diff(x, y)[0, :] takes the top row, the coefficients $a_0, \ldots, a_3 = [-2,\ 2,\ -1.4,\ 0.4857]$.
- newton_poly 从 $p = a_n$ 开始，每步 $p \leftarrow a_{n-k} + (x - x_{n-k})p$：嵌套（Horner）求值，x 可以是整个数组。
  > newton_poly starts at $p = a_n$ and repeats $p \leftarrow a_{n-k} + (x - x_{n-k})p$: nested (Horner) evaluation, and x may be a whole array.
- np.arange(–5, 2.1, .1) 共 71 个点，最后一点是 2.0。曲线过 4 个数据点，最大值 13.43 在 $x = -3$，在 $x \approx 0.91$ 处有极小值 -0.99。
  > np.arange(–5, 2.1, .1) gives 71 points ending at 2.0. The curve passes all 4 data points, peaks at 13.43 at $x = -3$ and has a minimum -0.99 near $x \approx 0.91$.

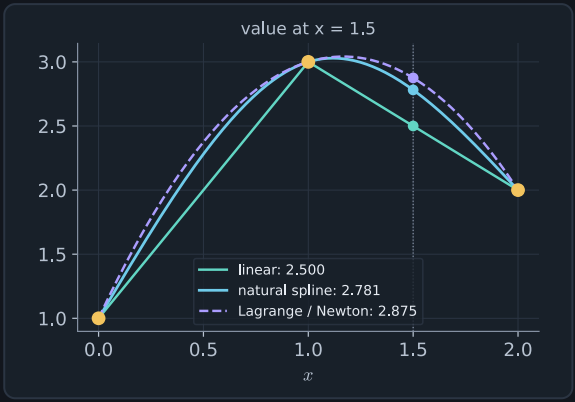
考点速记 KEY POINTS

- coef[i][j] $= f[x_i..x_{i+j}]$。
  coef[i][j] $= f[x_i..x_{i+j}]$.
- 系数取 coef[0, :]。
  Coefficients are coef[0, :].
- newton_poly 是 Horner 嵌套求值。
  newton_poly is nested (Horner) evaluation.
- 4 个点 → 三次多项式。
  4 points → a cubic.

7 四种方法的比较 拓展 EXT

7 Comparing the methods

方法 Method	形式 Form	光滑性 Smoothness
线性 Linear	分段直线 · piecewise lines	C^0：拐角 · corners
三次样条 Cubic spline	分段三次 · piecewise cubics	C^2
Lagrange	单个 $n-1$ 次多项式 · one polynomial	C^∞
Newton	同一多项式，便于加点 · same polynomial, easy to extend	C^∞



同一组数据 $x=[0,1,2]$、$y=[1,3,2]$ 在 $x=1.5$ 处：线性 2.500，自然样条 2.781，Lagrange/Newton 2.875。三者都满足插值条件，区别在于对点之间形状的假设。只有 3 个点时 not-a-knot 样条恰好就是这条二次多项式（也是 2.875）。

> For the same data $x=[0,1,2]$, $y=[1,3,2]$ at $x=1.5$: linear 2.500, natural spline 2.781, Lagrange/Newton 2.875. All three satisfy the interpolation condition; they differ in what they assume between the points. With only 3 points, the not-a-knot spline is exactly this quadratic (also 2.875).

多项式插值误差 / Polynomial interpolation error

$$f(x)-p_{n-1}(x)=\frac{f^{(n)}(\xi)}{n!}\prod_{i=1}^{n}(x-x_i),\qquad \xi \text{ between the nodes and } x$$

节点处乘积为 0；离节点越远（尤其区间外）乘积越大

/ the product is 0 at the nodes and grows away from them, especially outside the interval

情况 Situation	推荐 Choose
点很多、要快、可接受拐角 · many points, speed, corners acceptable	线性 · linear (np.interp)
要光滑曲线（轨迹、物理量）· need a smooth curve	三次样条 · cubic spline
点少（≤ 5）、需要解析多项式 · few points, need a formula	Lagrange / Newton
会不断增加数据点 · data points keep arriving	Newton
数据有噪声 · noisy data	回归，不用插值 · regression, not interpolation

考点速记 KEY POINTS

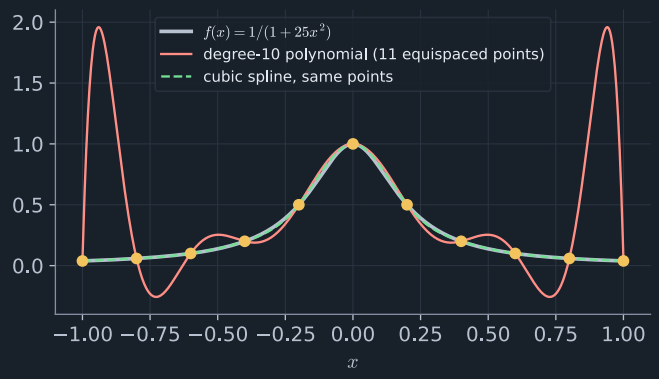
- $x=1.5$：线性 2.5，样条 2.781，多项式 2.875。
  $x=1.5$: linear 2.5, spline 2.781, polynomial 2.875.
- 误差含 $\prod(x-x_i)$，区间外迅速变大。
  Error has $\prod(x-x_i)$; it grows fast outside.
- 样条 C^2，线性只有 C^0。
  Spline is C^2; linear only C^0.
- 会加点用 Newton。
  Points keep arriving: Newton.

7.2 Runge 现象与外推风险 拓展 EXT

7.2 Runge's phenomenon and the risk of extrapolation

幻灯片提醒 Lagrange 多项式"只应在数据范围内使用"。更进一步：**在范围内**，高次多项式也可能很差。对 $f(x)=\frac{1}{1+25x^2}$ 取 $[-1,1]$ 上 11 个等距点做 10 次多项式插值：

> The slides warn that a Lagrange polynomial should be used only inside the data range. Going further: even **inside** the range a high-degree polynomial can be poor. Interpolate $f(x)=\frac{1}{1+25x^2}$ at 11 equally spaced points on $[-1,1]$ with a degree-10 polynomial:



多项式在两端剧烈振荡；同样点上的三次样条几乎与原函数重合

/ the polynomial oscillates wildly near the ends; a cubic spline on the same points nearly matches f

方法 Method	最大误差 Max error	$x=0.95$ 处 at $x=0.95$（真值 true 0.0424）
10 次多项式 · degree-10 polynomial	1.92（在 $x\approx\pm0.94$）	1.924
三次样条 · cubic spline	0.022	0.0436

外推 / Extrapolation

幻灯片数据的 $L(x)=-1.5x^2+3.5x+1$ 在区间外：$L(3)=-2$，$L(-1)=-4$，远离数据的取值范围 $[1,3]$。不同工具在区间外的行为也不同：interp1d 默认报错；np.interp 取端点值；CubicSpline 默认用端段的三次式继续外推。

> For the slide data, $L(x)=-1.5x^2+3.5x+1$ outside the interval gives $L(3)=-2$ and $L(-1)=-4$, far from the data range $[1,3]$. Tools also behave differently out of range: interp1d raises an error by default; np.interp returns the endpoint value; CubicSpline by default extends the end piece's cubic.

注意 / Caution

"点越多、次数越高越准"对等距节点的单个多项式不成立。补救：改用样条（分段低次），或改用在两端更密的 Chebyshev 节点。

> "More points and higher degree means more accurate" fails for a single polynomial on equally spaced nodes. Remedies: use a spline (low-degree pieces), or Chebyshev nodes, which cluster near the ends.

考点速记 KEY POINTS

- 等距高次插值：两端振荡（Runge）。
  Equispaced high degree: end oscillations (Runge).
- 外推 $L(3)=-2$：不可信。
  Extrapolated $L(3)=-2$: not trustworthy.
- 11 点：多项式误差 1.92，样条 0.022。
  11 points: polynomial error 1.92, spline 0.022.
- 补救：样条或 Chebyshev 节点。
  Fix: splines or Chebyshev nodes.

公式总表
Formula sheet

线性插值 / linear

$$\hat{f}(x) = y_i + \frac{(y_{i+1} - y_i)(x - x_i)}{x_{i+1} - x_i}$$

误差 $\leq \frac{h^2}{8} \max |f''|$ / error $\leq \frac{h^2}{8} \max |f''|$

三次样条 / cubic spline

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$

$n-1$ 段，$4(n-1)$ 未知数

/ $n-1$ pieces, $4(n-1)$ unknowns

样条条件 / spline conditions

$$S_i(x_i) = y_i,\ S_i(x_{i+1}) = y_{i+1};\ \ S_i', S_i'' \text{ continuous}$$

$2(n-1) + 2(n-2) + 2$ / $2(n-1) + 2(n-2) + 2$

自然边界 / natural boundary

$$S_1''(x_1) = 0, \qquad S_{n-1}''(x_n) = 0$$

Lagrange / Lagrange

$$L(x) = \sum_{i=1}^{n} y_i P_i(x), \quad P_i = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

$P_i(x_j) = \delta_{ij},\ \sum P_i = 1$ / $P_i(x_j) = \delta_{ij}, \sum P_i = 1$

Newton / Newton

$$f(x) = \sum_{k=0}^{n} a_k \prod_{m=0}^{k-1} (x - x_m), \quad a_k = f[x_0, \ldots, x_k]$$

差商递推 / divided differences

$$f[x_i..x_{i+j}] = \frac{f[x_{i+1}..x_{i+j}] - f[x_i..x_{i+j-1}]}{x_{i+j} - x_i}$$

插值误差 / interpolation error

$$f - p_{n-1} = \frac{f^{(n)}(\xi)}{n!} \prod_{i=1}^{n} (x - x_i)$$

当前版本 SciPy / NumPy 写法 / Current SciPy / NumPy calls

```
from scipy.interpolate import CubicSpline, BarycentricInterpolator
y_lin = np.interp(x_new, x, y)                  # linear; clamps outside range
f_nak = CubicSpline(x, y)                       # default bc_type='not-a-knot'
f_nat = CubicSpline(x, y, bc_type='natural')    # S''=0 at both ends
f_cla = CubicSpline(x, y, bc_type='clamped')    # S'=0 at both ends
f_lag = BarycentricInterpolator(x, y)           # replaces deprecated lagrange
```

考点速记 KEY POINTS

- 样条未知数 $4(n-1)$，方程同数。
  Spline: $4(n-1)$ unknowns, as many equations.
- Lagrange = Newton = 唯一插值多项式。
  Lagrange = Newton = the unique interpolant.
- Newton 系数 = 差商表第一行。
  Newton coefficients = top row of the table.
- CubicSpline 默认 not-a-knot；lagrange 已弃用。
  CubicSpline defaults to not-a-knot; lagrange is deprecated.

01 / 6 线性插值 · Linear

用线性插值估计温度

Estimating a temperature by linear interpolation

传感器在 $t = 0, 2, 4$ h 测得温度 $T = 10, 16, 13$ °C。用线性插值估计 $T(1)$ 和 $T(3)$。若在 $t = 5$ 调用 interp1d(t, T) 和 np.interp(5, t, T)，分别得到什么?

> **A sensor records $T = 10, 16, 13$ °C at $t = 0, 2, 4$ h. Estimate $T(1)$ and $T(3)$ by linear interpolation. What do interp1d(t, T) and np.interp(5, t, T) return at $t = 5$?**

答案 Answer 13.0 · 14.5 °C

$T(1) = 13.0$，$T(3) = 14.5$；$t = 5$ 时 interp1d 报错，np.interp 返回 13。

> **$T(1) = 13.0$, $T(3) = 14.5$; at $t = 5$ interp1d raises an error and np.interp returns 13.**

计算 / Computation

$$T(1) = 10 + \frac{(16-10)(1-0)}{2-0} = 10 + 3 = 13.0\ ^\circ\mathrm{C} \qquad (t \in [0,2])$$

$$T(3) = 16 + \frac{(13-16)(3-2)}{4-2} = 16 - 1.5 = 14.5\ ^\circ\mathrm{C} \qquad (t \in [2,4])$$

$t = 5$ 在数据范围 $[0, 4]$ 之外，属于外推：interp1d 默认 bounds_error=True，抛出 ValueError；np.interp 把范围外的值夹到端点，返回 $T(4) = 13$。两者都不是对 $t = 5$ 的可靠估计。

> $t = 5$ lies outside the data range $[0, 4]$, so this is extrapolation: interp1d defaults to bounds_error=True and raises ValueError; np.interp clamps to the endpoint and returns $T(4) = 13$. Neither is a reliable estimate at $t = 5$.

考试提醒 / Exam reminder

第一步永远是找对区间：$t = 3$ 要用 $(2, 16)$ 和 $(4, 13)$，不能用 $(0, 10)$。结果一定落在两端值之间（$13 \le 14.5 \le 16$），可用来自查。

> Step one is always the right interval: $t = 3$ uses $(2, 16)$ and $(4, 13)$, never $(0, 10)$. The answer must lie between the two end values ($13 \le 14.5 \le 16$), a quick self-check.

考点速记 KEY POINTS

- 先定区间，再代公式。
  Pick the interval first, then the formula.
- 线性插值结果在两端值之间。
  A linear estimate lies between the end values.
- interp1d 越界报 ValueError。
  interp1d raises ValueError out of range.
- np.interp 越界返回端点值。
  np.interp returns the endpoint value out of range.

02 / 6 Lagrange

手算 Lagrange 插值

Lagrange interpolation by hand

数据点 $(1,2)$、$(2,3)$、$(4,1)$。求三个 Lagrange 基多项式在 $x=3$ 的值，并求 $L(3)$。

> **Data points** $(1,2)$, $(2,3)$, $(4,1)$. **Evaluate the three Lagrange basis polynomials at** $x=3$ **and find** $L(3)$.

答案 Answer 8/3 ≈ 2.667

$P_1(3)=-\frac{1}{3},\ P_2(3)=1,\ P_3(3)=\frac{1}{3},\ L(3)=\frac{8}{3}$。

> $P_1(3)=-\frac{1}{3}, P_2(3)=1, P_3(3)=\frac{1}{3}, L(3)=\frac{8}{3}$.

推导 / Derivation

$$
\begin{aligned}
P_1(x) &= \frac{(x-2)(x-4)}{(1-2)(1-4)} = \frac{(x-2)(x-4)}{3} &&\Rightarrow\ P_1(3)=\frac{(1)(-1)}{3}=-\tfrac{1}{3}\\
P_2(x) &= \frac{(x-1)(x-4)}{(2-1)(2-4)} = -\frac{(x-1)(x-4)}{2} &&\Rightarrow\ P_2(3)=-\frac{(2)(-1)}{2}=1\\
P_3(x) &= \frac{(x-1)(x-2)}{(4-1)(4-2)} = \frac{(x-1)(x-2)}{6} &&\Rightarrow\ P_3(3)=\frac{(2)(1)}{6}=\tfrac{1}{3}
\end{aligned}
$$

$$
L(3)=2\left(-\tfrac{1}{3}\right)+3(1)+1\left(\tfrac{1}{3}\right)=\tfrac{8}{3}\approx 2.667, \qquad L(x)=-\tfrac{2}{3}x^2+3x-\tfrac{1}{3}
$$

自查：$P_1(3)+P_2(3)+P_3(3)=-\frac{1}{3}+1+\frac{1}{3}=1$ ✓；$L(1)=-\frac{2}{3}+3-\frac{1}{3}=2$ ✓。注意 $P_1(3)$ 为负：基多项式在节点之间可以是负数。

> Self-check: $P_1(3)+P_2(3)+P_3(3)=-\frac{1}{3}+1+\frac{1}{3}=1$ ✓; $L(1)=-\frac{2}{3}+3-\frac{1}{3}=2$ ✓. Note $P_1(3)$ is negative: basis polynomials can be negative between nodes.

考点速记 KEY POINTS

- 分母 = 分子在 x_i 处的值。
  Denominator = numerator evaluated at x_i.
- 自查：$\sum P_i(x)=1$。
  Self-check: $\sum P_i(x)=1$.
- $L(3)=8/3$。
  $L(3)=8/3$.
- 基多项式可以取负值。
  Basis values can be negative.

03 / 6 Newton · 差商 Divided differences

差商表与加入新点

Divided-difference table and adding a point

对 Q2 的数据 $(1,2)$、$(2,3)$、$(4,1)$ 建差商表，写出 Newton 多项式并求 $p_2(3)$。再加入第四个点 $(5,4)$，求新的 $p_3(3)$。

> **Build the divided-difference table for Q2's data $(1,2)$, $(2,3)$, $(4,1)$, write the Newton polynomial and find $p_2(3)$. Then add a fourth point $(5,4)$ and find the new $p_3(3)$.**

答案 Answer 8/3 → 5/3

$p_2(3)=\frac{8}{3}$（与 Q2 相同）；加点后只多一项 $a_3=0.5$，$p_3(3)=\frac{5}{3}$。

> $p_2(3)=\frac{8}{3}$ **(same as Q2); the new point adds one term** $a_3=0.5$**, giving** $p_3(3)=\frac{5}{3}$.

x_i	$f[x_i]$	一阶 first	二阶 second	三阶 third
1	**2**	$\frac{3-2}{2-1}=1$	$\frac{-1-1}{4-1}=-\frac{2}{3}$	$\frac{\frac{4}{3}-(-\frac{2}{3})}{5-1}=\frac{1}{2}$
2	3	$\frac{1-3}{4-2}=-1$	$\frac{3-(-1)}{5-2}=\frac{4}{3}$	
4	1	$\frac{4-1}{5-4}=3$		
5	4			

$$p_2(x)=2+1\cdot(x-1)-\tfrac{2}{3}(x-1)(x-2) \quad\Rightarrow\quad p_2(3)=2+2-\tfrac{4}{3}=\tfrac{8}{3}$$
$$p_3(x)=p_2(x)+\tfrac{1}{2}(x-1)(x-2)(x-4) \quad\Rightarrow\quad p_3(3)=\tfrac{8}{3}+\tfrac{1}{2}(2)(1)(-1)=\tfrac{5}{3}$$

$p_2(3)=\frac{8}{3}$ 与 Q2 的 Lagrange 结果相同，体现唯一性。加入 $(5,4)$ 时，原来的三个系数 $2,\ 1,\ -\frac{2}{3}$ 不变，只需算表的新一条斜线并追加一项：这就是幻灯片所说的"便于更新"。（Lagrange 则要重算所有基多项式。）

> $p_2(3)=\frac{8}{3}$ matches Q2's Lagrange result, showing uniqueness. Adding $(5,4)$ leaves the old coefficients $2,\ 1,\ -\frac{2}{3}$ unchanged; only the table's new diagonal is computed and one term appended, which is the "easy update" the slides mention. (Lagrange would have to recompute every basis polynomial.)

考点速记 KEY POINTS

- 二阶差商分母 $x_{i+2}-x_i$，不是 $x_{i+1}-x_i$。
  Second-order denominator: $x_{i+2}-x_i$, not $x_{i+1}-x_i$.
- 新项在所有旧节点处为 0。
  The new term vanishes at every old node.
- 加点：旧系数不变，追加一项。
  Adding a point: old coefficients stay, one term is appended.
- Newton 与 Lagrange 结果相同。
  Newton and Lagrange agree.

04 / 6 样条计数 · Spline counting

三次样条有多少个未知数

How many unknowns does a cubic spline have?

用自然三次样条插值 $n = 6$ 个数据点。需要求解多少个未知系数?

> **A natural cubic spline interpolates $n = 6$ data points. How many unknown coefficients must be solved for?**

A 24 个未知数
> 24 unknowns

B 20 个未知数 ✓ 正确 Correct
> 20 unknowns

C 18 个未知数
> 18 unknowns

D 10 个未知数
> 10 unknowns

答案 Answer B

6 个点 → 5 段，每段 4 个系数：$4(n-1) = 20$。

> **6 points → 5 pieces with 4 coefficients each: $4(n-1) = 20$.**

方程逐项计数 / Equation count

$$\underbrace{2(n-1) = 10}_{\text{interpolation}} + \underbrace{2(n-2) = 8}_{S',\, S'' \text{ continuity}} + \underbrace{2}_{S''=0 \text{ at ends}} = 20$$

干扰项为什么错 / Why the distractors are wrong

A $4n = 24$：把点数当成段数。n 个点只有 $n-1$ 段。
> $4n = 24$ counts points as pieces; n points give only $n-1$ pieces.

C 18 是不含边界条件的方程数 $(10 + 8)$：还差 2 个，所以必须补边界条件。
> 18 is the number of equations without boundary conditions $(10 + 8)$; 2 are missing, which is why boundary conditions are needed.

D 10 只是插值条件 $2(n-1)$ 的个数。
> 10 counts only the interpolation conditions $2(n-1)$.

考试提醒 / Exam reminder

"为什么需要边界条件？"标准答案：插值和连续性只给出 $4n - 6$ 个方程，比未知数 $4n - 4$ 少 2 个，两端各补一个。

> "Why are boundary conditions needed?" Standard answer: interpolation and continuity give only $4n - 6$ equations, 2 short of the $4n - 4$ unknowns, so one condition is added at each end.

考点速记 KEY POINTS

- 未知数 $4(n-1)$。
  Unknowns $4(n-1)$.
- 插值 $2(n-1)$，连续 $2(n-2)$。
  Interpolation $2(n-1)$, continuity $2(n-2)$.
- 少 2 个 → 边界条件 2 个。
  2 short → 2 boundary conditions.
- $n = 6$：$10 + 8 + 2 = 20$。
  $n = 6$: $10 + 8 + 2 = 20$.

05 / 6 自然样条手算 · Natural spline by hand

手算幻灯片数据的自然样条

The natural spline for the slide data, by hand

对 $x=[0,1,2]$、$y=[1,3,2]$ 求自然三次样条，并计算 $S(1.5)$。用局部形式 $S_1=a_1x^3+b_1x^2+c_1x+d_1$ $(x\in[0,1])$，$S_2=a_2t^3+b_2t^2+c_2t+d_2$ $(t=x-1)$。

> **Find the natural cubic spline for $x=[0,1,2]$, $y=[1,3,2]$ and compute $S(1.5)$. Use local forms $S_1=a_1x^3+b_1x^2+c_1x+d_1$ on $[0,1]$ and $S_2=a_2t^3+b_2t^2+c_2t+d_2$ with $t=x-1$.**

答案 Answer 2.78125

$S_1=-0.75x^3+2.75x+1$，$S_2=0.75t^3-2.25t^2+0.5t+3$，$S(1.5)=2.78125$。

> $S_1=-0.75x^3+2.75x+1$, $S_2=0.75t^3-2.25t^2+0.5t+3$, $S(1.5)=2.78125$.

8 个方程 / The 8 equations

插值（4 个） / interpolation (4)

$$S_1(0)=d_1=1$$
$$S_1(1)=a_1+b_1+c_1+d_1=3$$
$$S_2(0)=d_2=3$$
$$S_2(1)=a_2+b_2+c_2+d_2=2$$

连续（2 个）+ 自然边界（2 个） / continuity (2) + natural (2)

$$S_1'(1)=S_2'(0):\ 3a_1+2b_1+c_1=c_2$$
$$S_1''(1)=S_2''(0):\ 6a_1+2b_1=2b_2$$
$$S_1''(0)=0:\ b_1=0$$
$$S_2''(1)=0:\ 6a_2+2b_2=0$$

$$b_1=0,\ d_1=1\Rightarrow c_1=2-a_1;\quad b_2=3a_1;\quad a_2=-\tfrac{b_2}{3}=-a_1;\quad c_2=3a_1+c_1=2a_1+2$$
$$a_2+b_2+c_2=-1\ \Rightarrow\ -a_1+3a_1+2a_1+2=-1\ \Rightarrow\ a_1=-0.75$$
$$\Rightarrow\ c_1=2.75,\quad a_2=0.75,\quad b_2=-2.25,\quad c_2=0.5$$

$$S(1.5)=S_2(0.5)=0.75(0.125)-2.25(0.25)+0.5(0.5)+3=2.78125$$

注意 / Caution

本题用局部变量 t，系数与幻灯片的全局形式 $a_ix^3+b_ix^2+c_ix+d_i$ 不同，但曲线相同。与 SciPy 一致：CubicSpline(x, y, bc_type='natural') 的 c 属性正是 $[-0.75,0,2.75,1]$ 与 $[0.75,-2.25,0.5,3]$。

> This uses the local variable t, so the coefficients differ from the slide's global form $a_ix^3+b_ix^2+c_ix+d_i$, but the curve is the same. It matches SciPy: the c attribute of CubicSpline(x, y, bc_type='natural') is exactly $[-0.75,0,2.75,1]$ and $[0.75,-2.25,0.5,3]$.

考点速记 KEY POINTS

- 3 点 → 2 段 → 8 个未知数。
  3 points → 2 pieces → 8 unknowns.
- 局部形式：$S_i(x_i)=d_i$，$S_i''(x_i)=2b_i$。
  Local form: $S_i(x_i)=d_i$, $S_i''(x_i)=2b_i$.
- $S''(1)=-4.5$，$S'(1)=0.5$。
  $S''(1)=-4.5$, $S'(1)=0.5$.
- $S(1.5)=2.78125$。
  $S(1.5)=2.78125$.

06 / 6 Runge 现象 · Runge (EXT)

点越多，多项式插值就越准吗

Do more points always improve polynomial interpolation?

判断：对 $[-1,1]$ 上的光滑函数，使用越多的等距节点和越高次的单个插值多项式，区间内的插值误差总会越小。

> **True or false: for a smooth function on $[-1,1]$, using more equally spaced nodes and a single higher-degree interpolating polynomial always reduces the interpolation error inside the interval.**

答案 Answer 否 · No

$f=\frac{1}{1+25x^2}$、**11 个等距点：10 次多项式最大误差 1.92，比函数本身还大。**

> $f=\frac{1}{1+25x^2}$ **with 11 equispaced points: the degree-10 polynomial's maximum error is 1.92, larger than the function itself.**

推导 / Derivation

$$f(x)-p_{n-1}(x)=\frac{f^{(n)}(\xi)}{n!}\prod_{i=1}^{n}(x-x_i)$$

对 Runge 函数，$f^{(n)}$ 随 n 增长得比 $n!$ 还快；等距节点下 $\prod(x-x_i)$ 在两端附近也最大。两者相乘，误差在两端随 n 增大而发散。数值：$x=0.95$ 处真值 0.0424，多项式给出 1.924，三次样条给出 0.0436。

> For Runge's function, $f^{(n)}$ grows faster than $n!$, and with equispaced nodes $\prod(x-x_i)$ is also largest near the ends. Together the error diverges near the ends as n grows. Numbers: at $x=0.95$ the true value is 0.0424, the polynomial gives 1.924, the cubic spline 0.0436.

注意 / Caution

本题超出幻灯片（标拓展）。与幻灯片的联系：幻灯片警告 Lagrange 多项式在区间外会"不切实际地增长"；Runge 现象说明高次多项式在区间内的两端附近同样会出问题。

> This goes beyond the slides (extension). The link: the slides warn that the Lagrange polynomial shows unrealistic growth outside the interval; Runge's phenomenon shows high-degree polynomials also misbehave near the ends inside it.

考点速记 KEY POINTS

- 等距 + 高次 → 两端振荡。
  Equispaced + high degree → end oscillations.
- 11 点误差 1.92；样条 0.022。
  11 points: error 1.92; spline 0.022.
- 补救：样条或 Chebyshev 节点。
  Fix: splines or Chebyshev nodes.
- 误差公式含 $\prod(x-x_i)$。
  The error formula contains $\prod(x-x_i)$.

答案速查与来源

Answer key and sources

题号 Q	答案 Ans.	要点 Key idea
1	**13.0, 14.5 °C**	先定区间；越界：interp1d 报错，np.interp 取端点 · pick the interval; out of range: interp1d errors, np.interp clamps
2	$8/3$	$P_i(3) = -\frac{1}{3},\ 1,\ \frac{1}{3}$，和为 1 · they sum to 1
3	$8/3 \to 5/3$	加点只追加一项 $a_3 = \frac{1}{2}$ · a new point appends one term
4	**B**	$4(n-1) = 20$ = 10 + 8 + 2
5	**2.78125 ***	局部形式 8 个方程 · 8 equations in local form
6	**否 No ***	Runge 现象：误差 1.92 vs 样条 0.022 · Runge: 1.92 vs spline 0.022

带 * 的题超出幻灯片。幻灯片的两处错误（P_1 漏 $+1$；"之和"应为加权和）见第 5.2 节编者注。

> Starred items go beyond the slides. The two slide errors (P_1 missing $+1$; "sum" should be a weighted sum) are in the Section 5.2 editorial note.

来源 / Sources

1. PHYB57 C17 lecture slides, "Interpolation" (PHYB57_C17.pdf, 10 slides)
   用户上传 · uploaded by the user
2. Q. Kong, T. Siauw, A. Bayen, Python Programming and Numerical Methods, Ch. 17 Interpolation
   https://pythonnumericalmethods.berkeley.edu/notebooks/chapter17.00-Interpolation.html
3. Same book, §17.4 Lagrange Polynomial Interpolation (correct $P_1 = \frac{1}{2}(x^2 - 3x + 2)$)
   https://pythonnumericalmethods.berkeley.edu/notebooks/chapter17.04-Lagrange-Polynomial-Interpolation.html
4. SciPy documentation, CubicSpline (bc_type, default 'not-a-knot')
   https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.CubicSpline.html
5. SciPy documentation, lagrange (deprecated) · BarycentricInterpolator
   https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.lagrange.html
6. SciPy documentation, interp1d (legacy)
   https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.interp1d.html

图均按幻灯片代码用 Matplotlib 重新绘制；数值由 Python（SciPy 1.18.1，NumPy 2.5）复算。

> Figures were redrawn with Matplotlib from the slides' code; numbers were recomputed in Python (SciPy 1.18.1, NumPy 2.5).

考点速记 KEY POINTS

- 线性公式 + 样条计数 + Lagrange/Newton 构造必背。
  Memorise linear, spline counting, Lagrange/Newton construction.
- $P_1 = \frac{1}{2}x^2 - 1.5x + 1$（幻灯片有误）。
  $P_1 = \frac{1}{2}x^2 - 1.5x + 1$ (slide is wrong).
- 差商分母用两端 x。
  Divided differences: outer x in the denominator.
- 只在数据范围内插值。
  Interpolate only inside the data range.